CPE 314 — Software Design

University of Cebu

Computer Engineering Department

UC-B Campus Marketplace with Reviews

Project Design Document

Table 1: Group information

Group Name / No.:	Group 1 / Team 01 | BSCPE 3B | EDP 69323
Members:	John Benedict Alaiza
	Acexion B. Mahigugmaon
	Glyde V. Arreglado
	Tristan Dominic Urot
	Joshua Brian S. Enovero
	Eman De Paz
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026–2027
Document Version / Date:	V 0.2 / 25 October 2026 - revised design

Table of Contents

Revision History	3
1. Introduction & Project Overview	4
2. Preliminary ER Sketch & Team Roles	4
2.1 Key Attributes and Relationships	5
2.2 Team Roles	5
3. System Architecture & Object-Oriented Design	6
3.1 Runtime behavior and supporting classes	7
4. Data Model & Database Design	8
4.1 Subtype records and cardinalities	9
4.2 Schema description	10
5. API Integration Design	11
6. Concurrency Design	11
7. Data Mining & Visualization Design	11
8. Testing Summary	11
9. Individual Contributions	11
Appendix	12
Project References	12
Document Status	12
Milestone 1 preparation	13

Revision History

Table 2: Document revisions

Version	Date	Author	Changes
0.1	23 Oct 2026	Group 1	Initial Week 8 draft with introduction, preliminary ER sketch and team responsibilities.
0.2	25 Oct 2026	Group 1	Week 9 continuation: simpler implemented classes, one-screen GUI, matching schema and verified test results.

1. Introduction & Project Overview

UC-B Campus Marketplace with Reviews is a local Java desktop prototype for students, faculty and school groups to post and browse campus goods. It follows Option J with Textbook, Electronics and Furniture listings and their different condition rules. Milestone 1 uses JavaFX, SQLite/JDBC, category recursion, buyer-interest waitlists and recently viewed items.

Reviews, FakeStoreAPI suggestions, background JSON sales import, category analytics and charts remain planned for Milestone 2. Campus uniforms and department merchandise remain proposed extensions. The current prototype uses fictional demo members and local storage; authentication and online payments are outside this milestone.

2. Preliminary ER Sketch & Team Roles

The preliminary model carries forward the entities, key attributes and relationships from the proposal. The sketch below shows the core listing relationships; Table 3 lists all seven proposed records and their foreign keys.

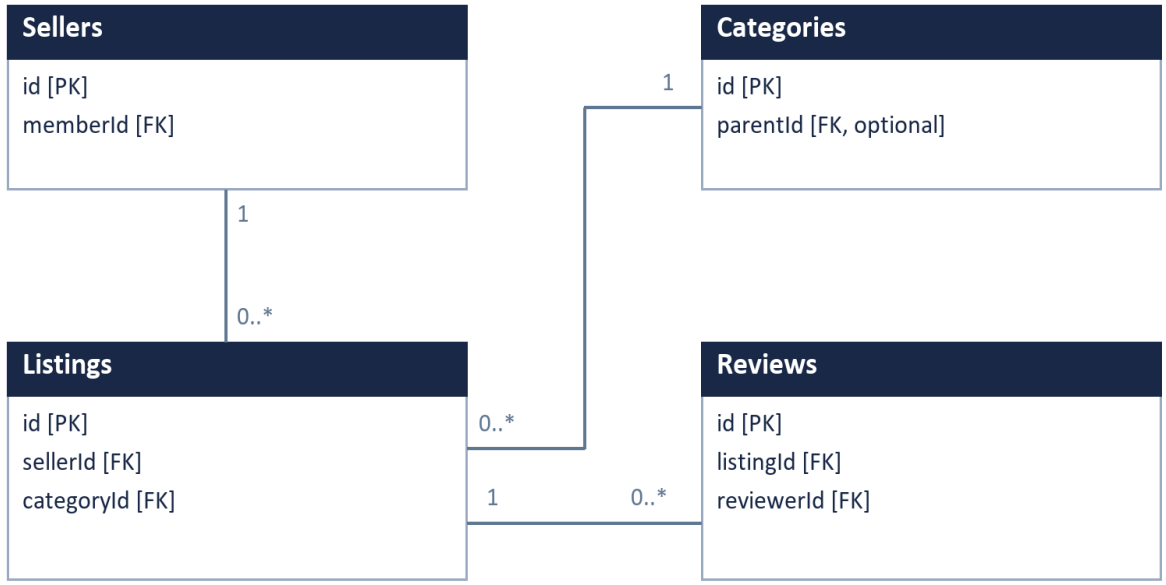


Figure 1: Core preliminary entity relationships

Each seller and category can relate to many listings; each listing can receive many reviews. The 1 and 0..* labels mean one parent and zero or more related records. This is a proposal sketch; the final ERD and actual schema will be documented in Section 4.

2.1 Key Attributes and Relationships

PK identifies a record uniquely. FK refers to a related record. Categories.parentId may be empty for a root category.

Table 3: Preliminary entities and key attributes

Entity	Key attributes
Members	id PK; name
Sellers	id PK; memberId FK → Members.id; displayName; sellerKind
Categories	id PK; parentId FK → Categories.id; name
Listings	id PK; sellerId FK → Sellers.id; categoryId FK → Categories.id; type; title; priceMinor; currency; status
Reviews	id PK; listingId FK → Listings.id; reviewerId FK → Members.id; rating; comment
BuyerInterests	id PK; listingId FK → Listings.id; buyerId FK → Members.id; joiningOrder
HistoricalSales	id PK; categoryId FK → Categories.id; sourceRecordId; priceMinor; currency; soldAt

The Week 8 sketch introduces the seven core records. Listings also have one matching subtype-details record. Section 4 gives the finalized fields, all ten tables and the complete relationships.

2.2 Team Roles

Table 4: Proposed primary responsibilities for Week 8

Member	Primary responsibility
John Benedict Alaiza - Backend Developer (Listing Models)	Sample listings, categories and condition examples.
Acexion B. Mahigugmaon - Project Leader / Integration	Architecture, implementation planning and integration.
Glyde V. Arreglado - Database Admin	ER sketch, attributes and relationship review.
Tristan Dominic Urot - UI/GUI Developer	GUI sketches, control labels and user steps.
Joshua Brian S. Enovero - Backend & API Developer	Sample JSON and API/import validation cases.
Eman De Paz - QA & Testing	Test cases and expected analytics calculations.

These are proposed responsibilities for member confirmation. Actual contributions and supporting commits will be recorded in Section 9.

3. System Architecture & Object-Oriented Design

The working Milestone 1 prototype has 12 Java files. Models hold item rules, a service coordinates actions, a database class handles SQL, and one JavaFX view displays the screen.

Listing (abstract)
- Textbook extends Listing
- Electronics extends Listing
- Furniture extends Listing

Listing stores the shared ID, seller/category IDs, title, price, currency and status. It declares getType() and gradeCondition(). Each subclass stores its condition evidence and implements these methods.

Table 5: Class responsibilities and SRP justification

Class	Single responsibility and reason to change
Listing	Stores and validates common item values and defines the grading contract. Changes when shared item rules change; it contains no SQL or GUI controls.
Textbook	Stores textbook evidence and grades its condition. Changes when textbook condition policy changes.
Electronics	Stores electronics evidence and grades its condition. Changes when electronics condition policy changes.
Furniture	Stores furniture evidence and grades its condition. Changes when furniture condition policy changes.

The table holds a List<Listing>. Calling gradeCondition() through that shared reference runs the actual subclass method. This is runtime polymorphism in the running app.

Implemented condition rules

Textbook: missing pages - POOR; otherwise heavy markings - FAIR; light markings or cover damage - GOOD; otherwise LIKE_NEW.

Electronics: no power or a functional defect - POOR; otherwise cosmetic damage - GOOD; otherwise LIKE_NEW.

Furniture: instability or structural damage - POOR; otherwise heavy wear - FAIR, light wear - GOOD, no wear - LIKE_NEW.

3.1 Runtime behavior and supporting classes

The other eight files support the four listing classes. Their jobs are grouped below.

Table 6: Supporting responsibilities outside the listing hierarchy

Component	Responsibility
App	Starts JavaFX, connects the database, service and screen, and closes the worker on exit.
MarketplaceView	Builds the screen, reads form values, handles button clicks and displays results.
MarketplaceService	Coordinates posting, member ownership, category recursion, waitlists and recent views.
Database	Opens SQLite, creates tables, runs bound SQL and saves/loads listing objects.
Member and Category	Hold member identity and category name/parent information.
ConditionGrade and WearLevel	Define the allowed condition grades and wear values.

Working GUI flow

The single screen has a listing table, inline posting form, category/search controls and details. Post listing saves a typed item and reloads the table with SELECT. View details records a view; Join waitlist adds buyer interest.

Data structures and recursion

Queue: Queue<Long> backed by ArrayDeque displays buyers in FIFO joining order. Saved rows rebuild the queue; offer() adds at the tail and poll() removes the head for display. Saved rows remain unchanged. Duplicate entries are rejected.

Stack: a Map stores one Stack<Long> per member. push() records a view; pop() reads a copy, newest first. History holds at most 20 unique items for the session; three are previewed.

Recursion: collectCategoryIds() calls itself for each child category. A leaf has no children and returns. Electronics includes Computers, Laptops and Gaming Laptops. Missing parents and cycles are rejected before traversal.

Build and background work

Java 21 and Maven provide shared libraries. Saved editor Run profiles launch App directly. One worker runs database operations; JavaFX displays the result on its UI thread.

4. Data Model & Database Design

The implemented SQLite schema follows the Week 8 sketch. Figure 2 shows seven core tables and Figure 3 shows the three subtype tables. Together they match the ten tables defined in schema.sql.

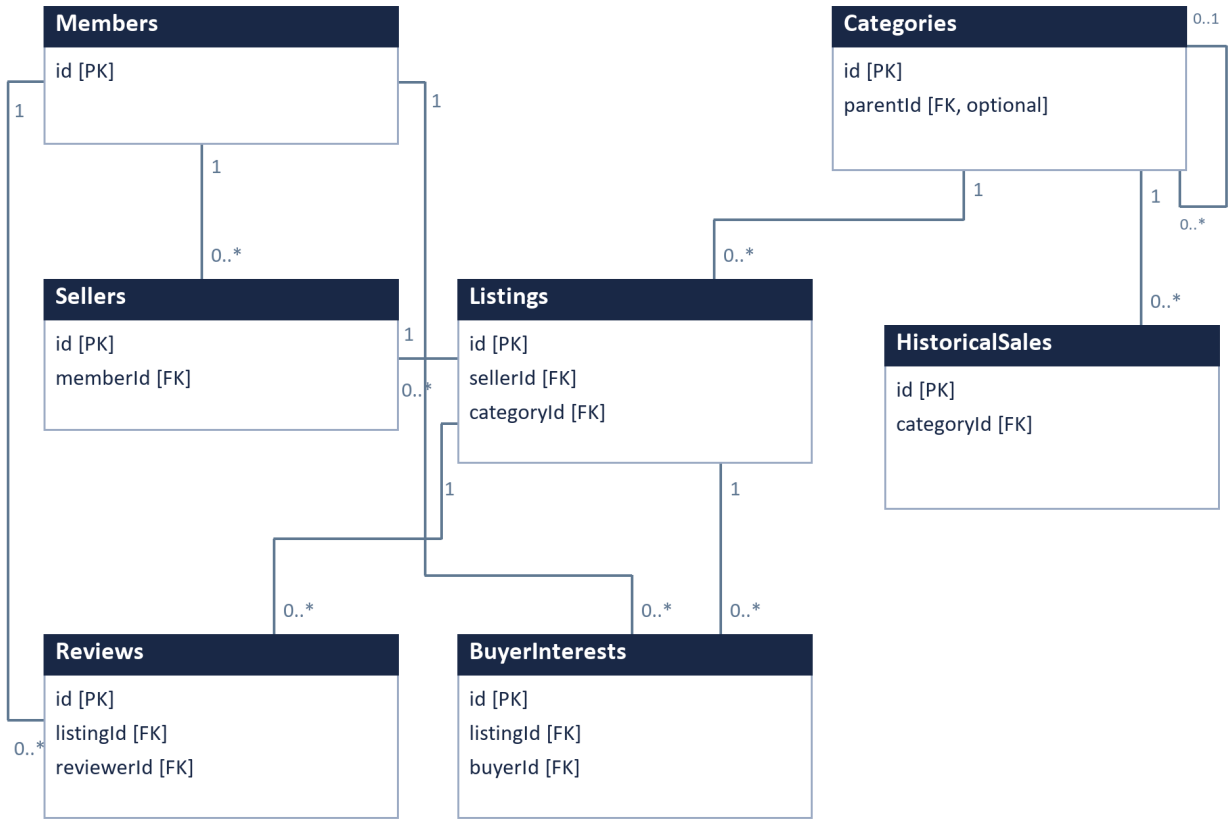


Figure 2: Week 9 core entity relationships

The 1 and 0..* endpoints mean one parent and zero or more child records. A root category has no parent; each non-root category has one parent. A parent can have many children. Every foreign-key relationship is listed explicitly on the following page.

4.1 Subtype records and cardinalities

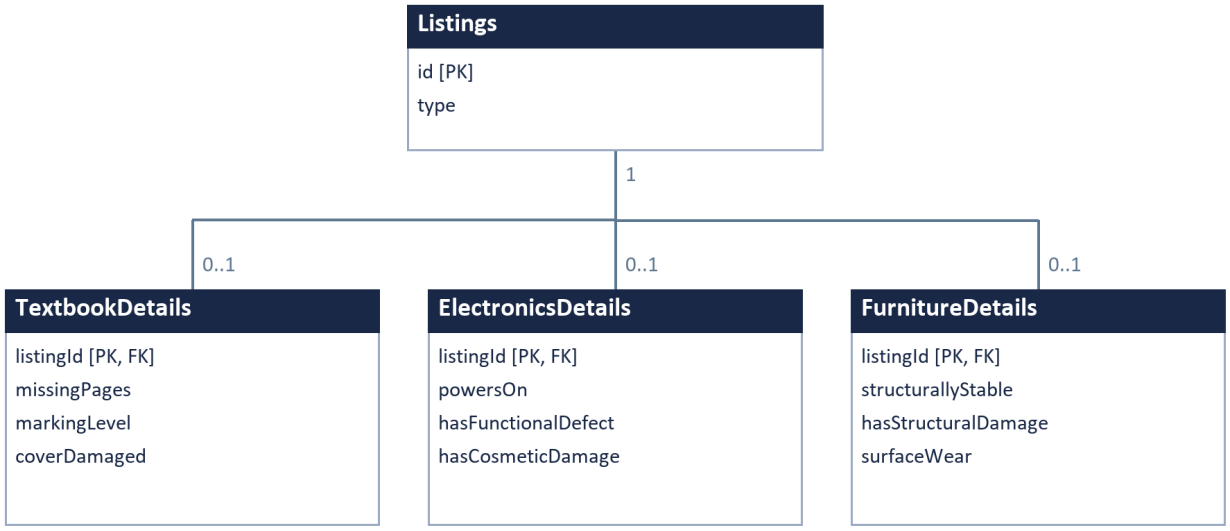


Figure 3: Listing subtype tables with shared primary keys

Each listing has one matching subtype-details row. Database.insertListing() saves both rows in one transaction and rolls back on failure. The load method checks that the detail row matches the listing type. Foreign keys alone do not enforce this rule.

Table 7: Complete relationship cardinalities

Parent	Child	Cardinality
Members	Sellers	1 to 0..*
Members	Reviews	1 to 0..*
Members	BuyerInterests	1 to 0..*
Sellers	Listings	1 to 0..*
Categories	Listings	1 to 0..*
Categories	HistoricalSales	1 to 0..*
Categories	Categories	0..1 parent to 0..* children
Listings	Reviews	1 to 0..*
Listings	BuyerInterests	1 to 0..*
Listings	Each subtype table	1 to 0..1; exactly one matching subtype overall

4.2 Schema description

This table matches src/main/resources/db/schema.sql. IDs are INTEGER; FK means foreign key. Required text values are nonblank. Prices use integer minor units plus a three-letter uppercase currency: PHP 350.50 is stored as 35050 and PHP.

Table 8: SQLite schema matching the implemented ERD

Table	Columns and constraints
Members	id PK; name required TEXT.
Sellers	id PK; memberId required FK to Members; displayName and sellerKind required TEXT.
Categories	id PK; parentId nullable FK to Categories; name required TEXT; parentId cannot equal id.
Listings	id PK; sellerId/categoryId required FKs; type is TEXTBOOK, ELECTRONICS or FURNITURE; title required; priceMinor positive INTEGER; currency three uppercase letters; status nonblank TEXT. Java supports AVAILABLE and WITHDRAWN.
Reviews	id PK; listingId/reviewerId required FKs; rating INTEGER from 1 to 5; comment nullable TEXT. Review entry is Milestone 2.
BuyerInterests	id PK; listingId/buyerId required FKs; joiningOrder positive INTEGER; UNIQUE(listingId, buyerId) and UNIQUE(listingId, joiningOrder).
HistoricalSales	id PK; categoryId required FK; sourceRecordId unique required TEXT; priceMinor positive INTEGER; currency three uppercase letters; soldAt nonblank TEXT. JSON import and timestamp validation are later work.
TextbookDetails	listingId PK/FK to Listings; missingPages nonnegative INTEGER; markingLevel is NONE, LIGHT or HEAVY; coverDamaged is 0/1.
ElectronicsDetails	listingId PK/FK to Listings; powersOn, hasFunctionalDefect and hasCosmeticDamage are each 0/1.
FurnitureDetails	listingId PK/FK to Listings; structurallyStable and hasStructuralDamage are each 0/1; surfaceWear is NONE, LIGHT or HEAVY.

Foreign keys are enabled on every connection. The three detail tables use ON DELETE CASCADE. The retained backend withdrawal method changes status and keeps related history; this smaller GUI has no withdrawal button.

Database.insertListing() commits the listing and its subtype together, or rolls back both. Java validates category cycles. SQL values use PreparedStatement bindings. Later category price calculations will keep currencies separate.

5. API Integration Design

Milestone 2 will use FakeStoreAPI comparable products to suggest an editable category and reference price range. Matching, currency handling, timeouts and a manual fallback remain to be implemented.

6. Concurrency Design

Current database actions run on one background worker. JavaFX updates controls after the task completes. Milestone 2 adds the background JSON importer and API requests, with validation, cancellation and failure handling.

7. Data Mining & Visualization Design

Milestone 2 will calculate best-rated category and average listing price by category, then display labeled bar charts. Reviews, imported sales and currency grouping must be handled consistently before these results are shown.

8. Testing Summary

On 27 October 2026, clean verify passed 27 JUnit 5 tests: 8 model, 10 database and 9 service tests, with zero failures, errors or skipped tests. Cases cover grading boundaries, deep recursion, FIFO/LIFO order, duplicates, invalid values, foreign keys, save/reopen and transaction rollback.

The reduced GUI passed 23 checks, including Post listing, Search / refresh, View details, Join waitlist and validation feedback. Direct Java startup also passed. The live presentation rehearsal in Eclipse and Antigravity still needs verification.

9. Individual Contributions

The role and review assignments on pages 5 and 13 guide the team. Individual changes and explanations will be recorded from actual repository commits and reviews before submission. Peer ratings are completed privately by the members.

Appendix

This V0.2 revision continues the Week 9 design. Sections 3 and 4 describe the implemented class hierarchy, responsibilities, ERD and SQLite schema. The original proposal context and role assignments are retained.

Project References

Project repository

https://github.com/XionAce/cpe314-69323-capstone-team-01

Project proposal

https://github.com/XionAce/cpe314-69323-capstone-team-01/blob/project/proposal/docs/proposal.md

Student Guide — Group Capstone Project

https://drive.google.com/file/d/195Y8cNaHnd0M7FpIwmKDYzfok5pbKZti/view

Week 8 design-document checklist

https://drive.google.com/file/d/1wb9xb2nd7RDowEEbSMx-M-l58qnO9gMM/view

Project design-document template

https://drive.google.com/file/d/1sRJ5rjciHBeYgBQmUbpJB12kWyD_sCkO/view

FakeStoreAPI documentation

https://fakestoreapi.com/docs

Week 9 milestone grading checklist

https://drive.google.com/file/d/1YWqldux-sU5sxi89t_Ue12gP6t2eWzQS/view

Week 9 design document checklist

https://drive.google.com/file/d/1WaBE_8WWOV7OQbFup8UoNtF0XXBH8CKz/view

Milestone 1 demo rubric

https://drive.google.com/file/d/11qO_eQ344vlPZwqgJmHdl9J1wNm58j9u/view

Document Status

Instructor approval was confirmed on 25 October 2026. The simplified prototype has Java source, a Maven build, a working JavaFX screen and a SQLite database.

Verification on 27 October recorded 27 passing JUnit tests and 23 passing GUI checks. Direct Java startup was also checked. The live Eclipse/Antigravity presentation rehearsal remains to be verified.

The checkpoint PR, member explanations and private peer evaluations still need final review. API suggestions, background JSON import, review entry, analytics and charts remain Milestone 2 work.

Milestone 1 preparation

The Week 9 checkpoint is worth 15% of the Finals Term Grade. Its design portion is 25 of 100 points: hierarchy description 7, class-by-class SRP 6, finalized ERD 7, matching SQLite schema 5. These written sections do not replace the live application demonstration.

Live minimums: an abstract/base type with at least two concrete subclasses invoked polymorphically; purposeful Stack, Queue or Map usage plus genuine recursion with a base case; a SQLite file with at least three tables; parameterized JDBC INSERT and SELECT; one working GUI event handler; at least two passing JUnit 5 tests including a nontrivial edge case. The full project retains all three assigned listing types and the planned six or more tests.

Table 9: Proposed milestone coding tasks

Member	Proposed task
Acexion B. Mahigugmaon - Project Leader / Integration	abstract Listing, common validation, build configuration and integration.
John Benedict Alaiza - Backend Developer (Listing Models)	concrete listing types and condition behavior.
Glyde V. Arreglado - Database Admin	schema initialization and parameterized repository INSERT/SELECT.
Tristan Dominic Urot - UI/GUI Developer	JavaFX posting/browsing window and working event handler.
Joshua Brian S. Enovero - Backend & API Developer	buyer-interest Queue and recursive category search.
Eman De Paz - QA & Testing	meaningful JUnit tests including an edge case, and live test-run rehearsal.

Every member commits and understands their own work. Balanced contributions and individual code explanations are assessed. The group/shared score is out of 85; private peer evaluation contributes 15 per member, calculated as (average received rating / 5) × 15. No scores or completed contributions are claimed here.

Submission: create project/milestone-1 from an up-to-date main, use personal feature branches into it, then open a PR into main titled exactly Milestone 1 — Team 01 before the class deadline. Leave the checkpoint PR open for instructor review; do not merge it yourselves. A calendar deadline is not specified in the supplied files.

Five-minute demo: hierarchy 1 minute; working GUI control 1 minute; database and live query 1.5 minutes; live JUnit tests 1 minute; targeted member question 0.5 minute. Full API integration, background JSON import, mining, charts and polish follow in Milestone 2.